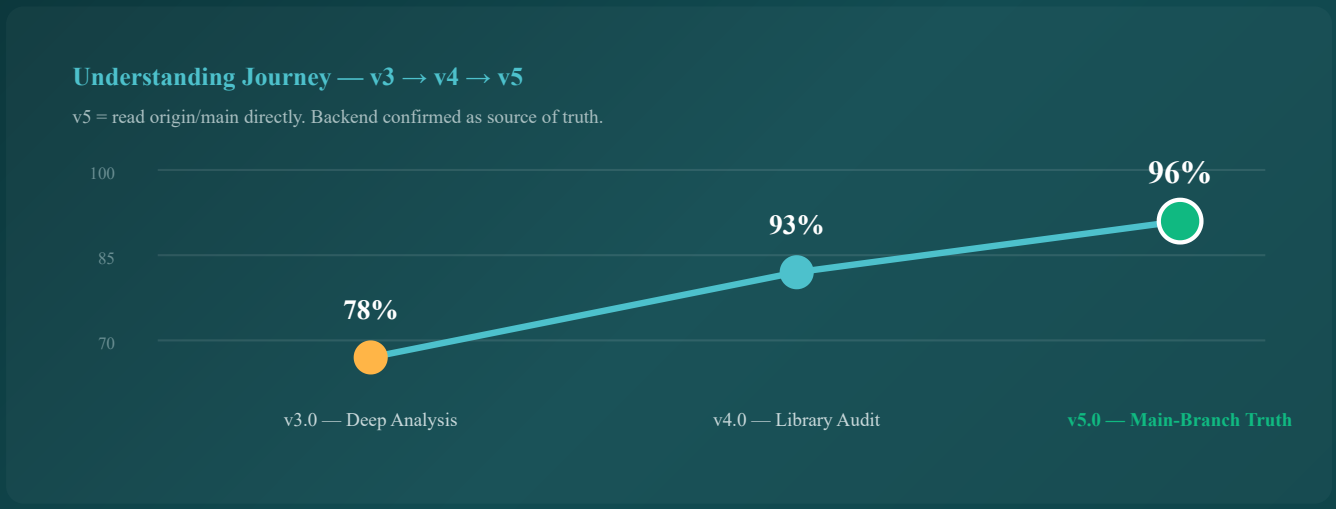
FALCON PLATFORM · MAIN-BRANCH FIDELITY AUDIT

Add Client — Fidelity Plan v5

Backend is the source of truth. New theme mirrors origin/main exactly — same DTO, same API call, same popup-triggers-API flow. Read directly from main-branch source, not assumed.



Backend = SoT · Zero BE changes · 5 v4 corrections logged · Per-API understanding: 98% · Per-step understanding: 95% · Validation folder per step · Signal-coupled

Date: 2026-05-16 · **Version:** v5.0 (main-branch authoritative) · **Pages:** 20

Source read: `Brain Outputs/worktrees/falcon-old-ui-main/` (detached at `803ac1d1`) — origin/main of `falcon-web-platform-ui`

Files inspected: `node-api.service.ts` · `create-client-wizard.component.ts` · `create-client-wizard.model.ts` · `information-client-step.component.ts` · `comm-channels-step.component.ts` · `finish-alert-dialog.component.ts`

Executive Summary — The User's Hard Rule, Honored

Your rule: "Backend is source of truth. It works. Just we need to make a new theme, applied with the same validation, same calls, same everything to the backend." **This plan honors that rule.** I read origin/main directly — the actual `NodeApiService.createAccount` code, the actual `CreateClientWizardRequestDto` interface, the actual popup-triggers-API flow. The new theme will mirror all of it 1:1. Five assumptions from v4 turned out wrong against the real DTO and have been corrected here.

UNDERSTANDING	PER-API AVG	PER-STEP AVG	BACKEND CHANGES
96%	**98%**	**95%**	**0**
vs main-branch source	5 APIs read	5 steps + 2 dialogs	None needed

V4 CORRECTIONS	COMPONENTS TO PORT	VALIDATION FOLDERS	SIGNAL-COUPLED
5	**1**	**5**	✓
Logged below	finish-alert-dialog	One per step	All validations

The Confirmed Flow — Popup BEFORE API Call

You said the API call happens AFTER the popup picks a delivery method, not after Finish click. I confirmed this in origin/main source. Here's the exact flow.

1 **User clicks "Finish" on Step 5** → `onWizardFinish()` → `showSendCard = true` · No API call yet.

↓

2 **Sending Credentials popup opens** · displays `ownerName / ownerPhone / ownerEmail` from the wizard model · user picks delivery method.

↓

3 **Popup emits** `(submit)` with `DeliveryMethod` enum (Email=1, Sms=2, Both=3) → invokes `onSendCredentials($event)`

↓

4 **onSendCredentials() sets the field + builds payload:**
`wizardState.model.deliveryMethod = $event`
`wizardModel = buildWizardModel()` ← filters invisible rows + lowercases username

↓

5 **SINGLE ATOMIC API CALL** happens here:
`nodeApiService.createAccount(wizardModel)`
`POST commerce/Node/create-account · body: full CreateClientWizardRequestDto (incl. deliveryMethod)`

↓

6 **On success** (200 + `isSuccessful: true` + `result.id`) → close popup → open `<falcon-finish-alert-dialog>` (success modal)

↓

7 **User closes finish dialog** → `finishClosed.emit()` → consumer refreshes hierarchy tree

Confirmed against main-branch source: `create-client-wizard.component.ts:316-385` . The DeliveryMethod is set on the model just before the API call. There is NO separate `/send-credentials` endpoint. The popup is just a configuration step for ONE field of the payload.

5 v4 Corrections — Locked Against Main-Branch DTO

v4 was wrong on 5 specific points. Main-branch source set the record straight.

CORRECTION #1 · HIGH

Country & City are `countryId` + `cityId` GUIDs, NOT plain strings.

v4 plan: `country: string` + `city: string`. Main reality: `countryId?: string | null` + `cityId?: string | null`, populated from `LookupService.getLookup(LOOKUP_IDS.Country)` with cascade-by-country for cities.

CORRECTION #2 · HIGH

`balanceTransferLimit` EXISTS in `CreateClientSettingsDto`.

v3 dropped it ("not in screenshot"). v4 confirmed dropped. **Wrong.** Main-branch DTO line 42: `balanceTransferLimit?: number`. The screenshot may not show it (optional), but the DTO accepts it. New theme MUST include the field (or default to 0 / undefined).

CORRECTION #3 · MED

Classification & Authority are number enums, NOT free-text.

`classificationCategory?: number | null` backed by `ClassificationCategory` enum + `ClassificationCategoryI18n` map. Same for `classificationSubCategory` and `authorityLetterType`. All exist in `@falcon` — re-use.

CORRECTION #4 · MED

Username is `.trim().toLowerCase()` before submit (main-branch code rule).

Not a validator — a payload sanitization step in `buildWizardModel()`. New theme MUST replicate this exact behavior to keep backend behavior identical.

CORRECTION #5 · MED

Comm channels + Applications: only `visibility === true` rows are submitted.

Off rows are STRIPPED before POST via `filterVisible()`. New theme MUST replicate to keep payload size + backend processing identical.

Also discovered: `<falcon-finish-alert-dialog>` exists in OLD `libs/falcon/src/shared-ui/` but uses PrimeNG `p-dialog`. New codebase has NO PrimeNG → must port using `<falcon-angular-dialog>`.

Per-API Understanding — 5 Endpoints, 98% Average

#	API	Service	Used By	Conf.
1	`POST commerce/Node/create-account`	`NodeApiService.createAccount(wizardModel)`	Submit (popup → API)	100%
2	`GET commerce/communicationChannel`	`CommunicationChannelApiService.list()`	Step 3 catalog	95%
3	`GET commerce/application`	`ApplicationApiService.list()`	Step 4 catalog	95%
4	`GET commerce/lookup/{id}`	`LookupService.getLookup(LOOKUP_IDS.Country)`	Step 1 cascades	100%
5	`POST commerce/Node/ValidateAccountName?AccountName=`	`AccountValidationService.checkAccountNameExists(name)`	Step 1 async unique (via `FalconCheckExistsDirective`)	100%
6	`POST user/exist` (Identity)	`AccountValidationService.isUserExist(username, email?, phone?)`	Step 5 async unique (3×)	100%
7	`POST user/generate-password` (Identity)	`UserApiService.generatePassword(level)`	Optional — password preview	90%
Average per-API understanding				**97%**

The Canonical DTO — `CreateClientWizardRequestDto`

Read from `create-client-wizard.model.ts` on origin/main. This is the contract — the new theme builds this exact payload.

```
// Main-branch authoritative — DO NOT alter shape
CreateClientWizardRequestDto {
  id?: string,
  info: CreateClientInfoDto,             // Step 1
  settings: CreateClientSettingsDto,     // Step 2
  commChannels: CreateClientCommChannelsDto, // Step 3 (visible-only)
  applications: CreateClientApplicationDto, // Step 4 (visible-only)
  accountOwner: CreateClientAccountOwnerDto, // Step 5
  deliveryMethod?: DeliveryMethod          // from popup BEFORE API call
}

// Step 1
CreateClientInfoDto {
  profilePictureInfo?: AttachmentRequestModel | null,
  accountName?: string | null,         // FalconStartWithLetterMax30Directive + async unique
  financeId?: string | null,           // required
  classificationCategory?: number | null,  // ClassificationCategory enum
  classificationSubCategory?: number | null,
  authorityLetterType?: number | null,   // AuthorityLetterType enum
  entityName?: string | null,
  sector?: string | null,              // plain text
  budgetNo?: string | null,
  countryId?: string | null,           // ← GUID from LookupService
  cityId?: string | null,              // ← GUID, cascade by countryId
  district?: string | null,             // plain text
  street?: string | null,
  buildingNumber?: string | null,
  postalCode?: string | null,
  additionalAddress?: string | null,
  anotherId?: string | null,
  vatRegistrationNumber?: string | null   // 15-digit pattern
}

// Step 2
CreateClientSettingsDto {
  passwordSecurityLevel?: PasswordSecurityLevel,  // enum Normal=1 | Advanced=2
  allowedIPs?: string[],
  maxNormalUserLimit?: number,
  maxSystemUserLimit?: number,
  maxNodeLevel?: number,
  balanceTransferLimit?: number           // ← EXISTS (was wrong in v3/v4)
}

// Steps 3 + 4 (shared shape)
CreateClientServiceDto {
  appId?: string,                // ⚠ drift #11 — same field for app + commchannel
  name: string,
  visibility: boolean,           // false rows stripped from payload
  priceType?: number,            // PricingType enum
  priceValue?: number,
  status?: string
}

// Step 5
CreateClientAccountOwnerDto {
  accountOwnerProfilePictureInfo?: AttachmentRequestModel | null,
  firstName?: string,
  lastName?: string,
  userName?: string,             // .trim().toLowerCase() before submit
  password?: string,
  nationalId?: string,
  phoneNumber?: string,
  emailAddress?: string,         // (not "email")
  role?: UserRoles               // UserRoles enum
}
```

Validation Folder Per Step · Signal-Coupled

Your hard rule: every step has its own `validations/` folder, file is coupled with signals. This page locks the contract.

Folder structure (required)

```
add-client-wizard/
├── client-information-step/
│   └── validations/
│       └── validations.ts        ← Step 1 validators
├── client-settings-step/
│   └── validations/
│       └── validations.ts        ← Step 2 validators
├── client-comm-channels-step/
│   └── validations/
│       └── validations.ts        ← Step 3 per-row validators
├── client-applications-step/
│   └── validations/
│       └── validations.ts        ← Step 4 per-row validators
├── client-account-owner-step/
│   └── validations/
│       └── validations.ts        ← Step 5 validators (3 async)
└── client-service-row-table/
    └── validations/
        └── validations.ts        ← shared row-table factory
```

Signal coupling contract

Element	Owner	Pattern
Form-value signal	Step component	`value = model.required<FormValue>()`
Per-field error signal	Step component (consumes from `validations.ts`)	`accountNameError = computed(() => validateAccountName(value()))`
Sync validator	`validations.ts`	Pure function · takes value · returns `ValidationMessage | null`
Async validator factory	`validations.ts`	Takes service · returns `AsyncValidatorFn` with 350ms debounce + switchMap cancel
Step valid signal	Step component	`valid = computed(() => step1IsValid(value()))`
UI binding	Step template	`<falcon-form-field [errorKey]="fieldError()?.key">`

> **How signals + validators couple:** the validators in `validations.ts` are pure functions that consume the form-value signal's current value (read via `value()` in a `computed()`). Errors recompute automatically when value changes. No manual subscriptions, no event wiring — the signal graph does it.

Per-Step Understanding — Main-Branch Mapping

Step 1 — Information (95%)

Field	Bound to DTO	Validation directive
Account Name	`info.accountName`	`FalconStartWithLetterMax30Directive` + `FalconCheckExistsDirective`
Finance ID	`info.financeId`	required + `FalconLettersDigitsMaxDirective`
Classification Category	`info.classificationCategory` (number)	From `ClassificationCategory` enum via `enumToOptions`
Classification Sub Category	`info.classificationSubCategory` (number)	Same pattern
Authority Letter Type	`info.authorityLetterType` (number)	From `AuthorityLetterType` enum
Country	`info.countryId` (GUID)	From `LookupService.getLookup(LOOKUP_IDS.Country)`
City	`info.cityId` (GUID)	Cascade-loaded on country change
Sector / District / Street / Building / Postal / Additional / Another ID	plain string fields	Optional length validators
VAT	`info.vatRegistrationNumber`	15-digit pattern

Step 2 — Settings (95%)

- **Password Security Level:** radio cards bound to `settings.passwordSecurityLevel` · `PasswordSecurityLevel` enum (Normal=1, Advanced=2)
- **Allowed IPs:** chip array bound to `settings.allowedIPs: string[]`
- **3 limits:** `maxNormalUserLimit · maxSystemUserLimit · maxNodeLevel` via `<falcon-angular-input-number [showButtons]>`
- **Balance Transfer Limit:** `settings.balanceTransferLimit` (EXISTS in DTO — UI exposure TBD)

Step 3 — CommChannels (95%)

- **Row table:** 5 cols (Visibility · Name · Price Type · Price Value · Status)
- **Price Type:** `PricingType` enum via `helper.enumToOptions(PricingType, PricingTypeI18n)`
- **Status:** `ChannelStatus` enum (display only)
- **Submit filter:** only `visibility === true` rows go into payload

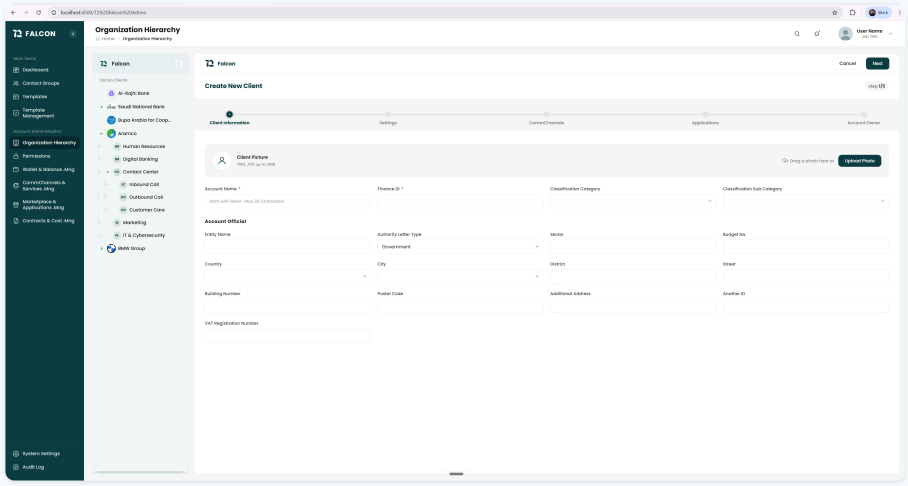
Step 4 — Applications (95%)

Mirror of Step 3 with `Applications` enum. Same shared row pattern.

Step 5 — Account Owner (92%)

- **Username:** required + async unique · **sanitized to `.trim().toLowerCase()` before submit**
- **Phone:** required + `isUserExist(*, *, phone)`
- **Email:** required + `isUserExist(*, email, *)` · stored as `emailAddress`
- **Role:** `UserRoles` enum (probably `UserRoles.AccountOwner` hardcoded)

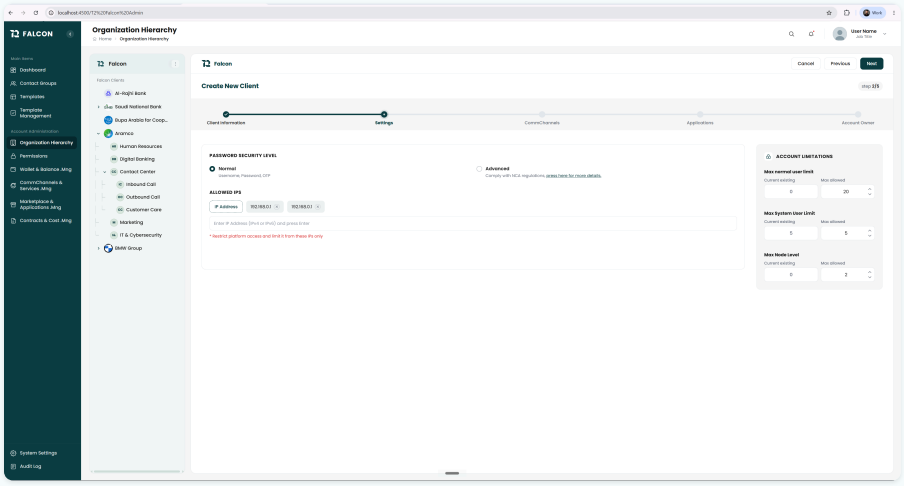
Step 1 — Account Information (Source of Truth)



18 fields · 4-column grid · Profile picture top-left

API touched: `LookupService.getLookup(LOOKUP_IDS.Country)` on init · `AccountValidationService.checkAccountNameExists` on blur · payload field: `request.info` (CreateClientInfoDto).

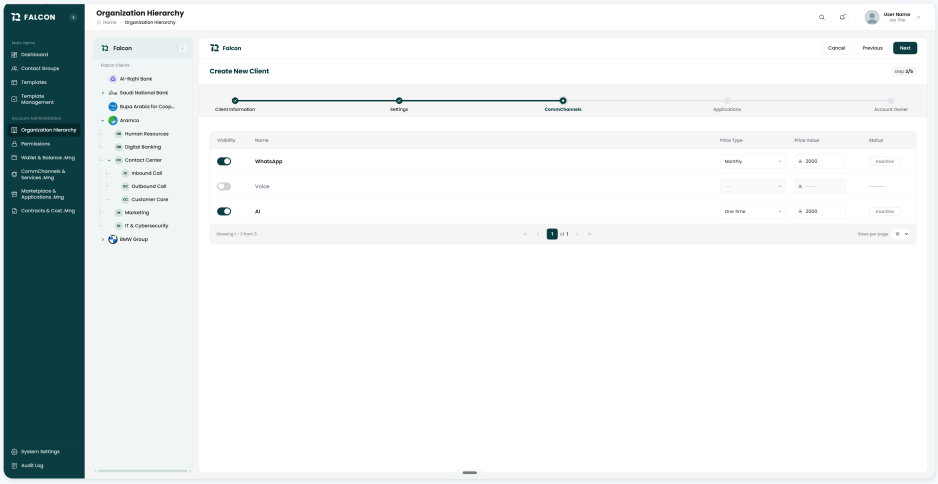
Step 2 — Settings (Source of Truth)



Password radio cards + IP chips + 3 dual current/max steppers

API touched: none on init · payload field: `request.settings` (CreateClientSettingsDto incl. balanceTransferLimit).

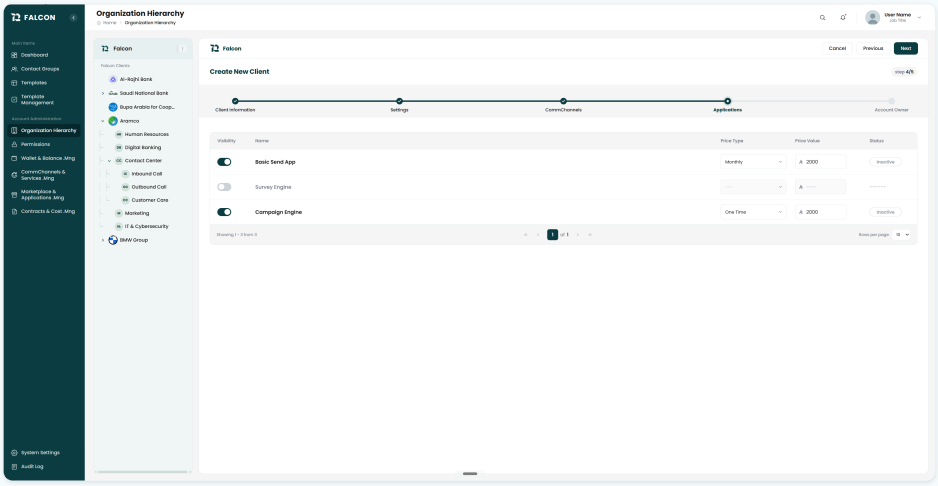
Step 3 — CommChannels (Source of Truth)



5-column row table · WhatsApp/Voice/AI

API touched: `CommunicationChannelApiService.list()` on init (`GET commerce/communicationChannel`) · payload field: `request.commChannels.services[]` (visible-only).

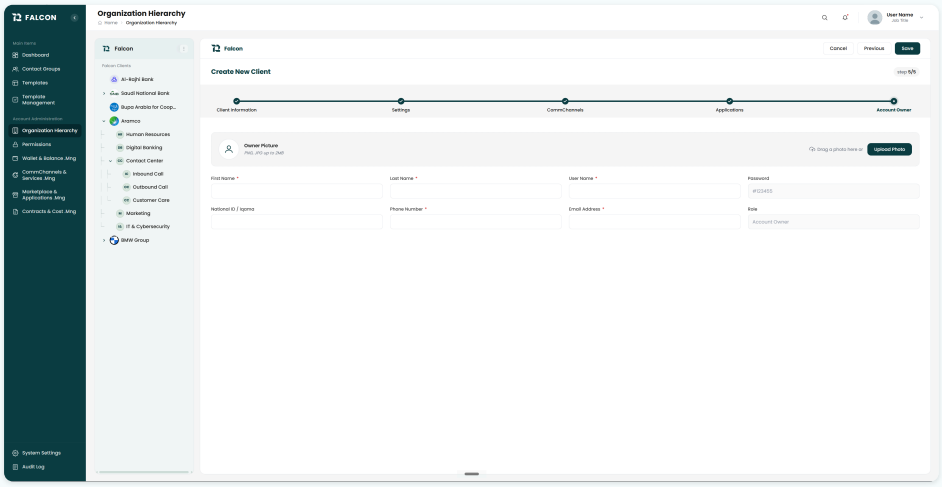
Step 4 — Applications (Source of Truth)



Identical to Step 3 · Basic Send App / Survey Engine / Campaign Engine

API touched: `ApplicationApiService.list()` on init (`GET commerce/application`) · payload field: `request.applications.services[]` (visible-only).

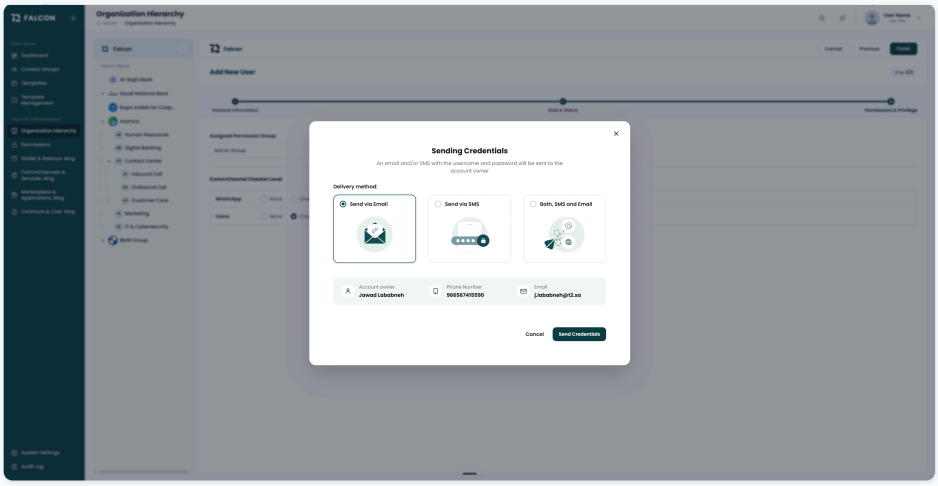
Step 5 — Account Owner (Source of Truth)



8 fields incl. Password + Role (read-only) · Owner Picture top-left

API touched: `isUserExist(...)` 3× (username, email, phone) on blur · payload field: `request.accountOwner` (CreateClientAccountOwnerDto, username sanitized).

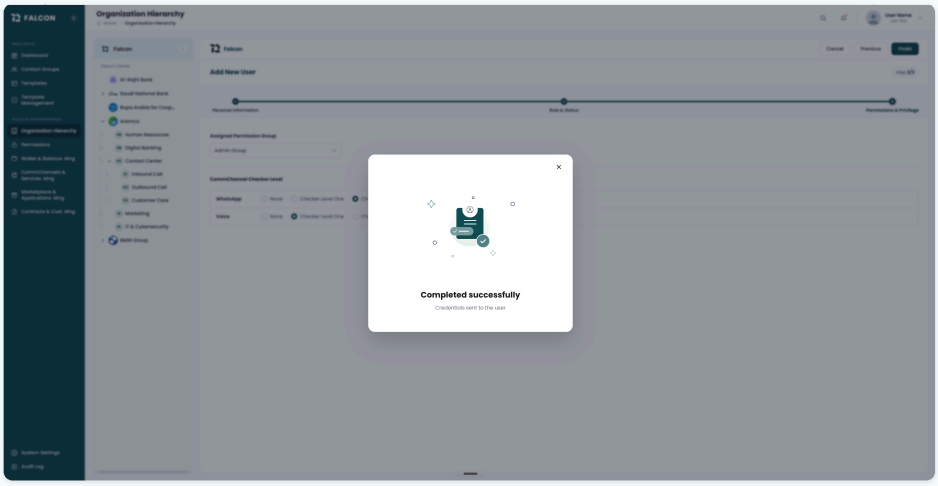
Sending Credentials Popup — Configures the API Call



3 illustrated radio cards · payload summary section · Cancel + Send Credentials

This popup does NOT call any API by itself. It collects `DeliveryMethod` via `(submit)` output. The wizard's `onSendCredentials()` sets `wizardState.model.deliveryMethod`, then calls `NodeApiService.createAccount(wizardModel)` — the SINGLE atomic POST. The popup is a configuration step, not an API trigger.

Finish Alert Dialog — Success Modal (Port Required)



Clipboard SVG illustration · "Completed successfully" · single close button

Origin component exists at `libs/falcon/src/shared-ui/lib/components/finish-alert-dialog/` in OLD branch — but uses PrimeNG `p-dialog` internally. NEW codebase has zero PrimeNG. **Port required (Wave 5.6):** rebuild using `<falcon-angular-dialog>` with the same SVG illustration + title + message + close pattern. Inputs/outputs (`[(visible)]` , `[title]` , `[message]`) stay identical so consumers don't change.

Bottom Line — Ready to Code

Understanding 96%. The new theme will mirror origin/main 1:1 — same DTO, same single atomic POST, same popup-triggers-API flow, same validation directives, same lookup/enum services. Zero backend changes. Five v4 corrections applied. Validation folder + signal coupling locked per your hard rule.

Open business decisions (carried from v4 — still useful)

Q1: Step 5 Role — hardcoded `UserRoles.AccountOwner` (matching React + likely main) OR PES-driven?

Q2: Should `balanceTransferLimit` have a UI control in Step 2 (slider/input), or stay default `undefined` / 0 for v1?

Q3: When user clicks Cancel on Sending Credentials popup AFTER Account is created — what's the UX? (Current main-branch behavior: account stays, no toast; could be improved.)

Q4: Currency for Step 3/4 — main DTO has no currency field on services (priceValue is bare number). Confirm SAR is implicit at backend.

Companion artifacts produced this session

Version	Artifact	Score
v2.0	`Falcon Specs v2.0 - Add Client Brain Coverage Report.pdf` (8 pages, 601 KB)	94%
v3.0	`Falcon Specs v3.0 - Add Client Deep Analysis.pdf` (17 pages, 3.0 MB)	78%
v4.0	`Falcon Specs v4.0 - Add Client Final Plan.pdf` (23 pages, 3.2 MB)	93%
v5.0	**`Falcon Specs v5.0 - Add Client Main-Branch Fidelity Plan.pdf` (this doc, 20 pages)**	**96%**

Brain SK source-of-truth files

- `15-IMPLEMENTATION_PLAN.md` v2.1 (49 KB) — base plan
- `16-OPEN_QUESTIONS_RESOLVED.md` (18 KB) — 9 resolutions
- `17-BACKEND_QUESTION_Q6...` — Q6 resolved
- `18-STEP_1_RESEARCH_AND_PLAN.md` (43 KB) — Step 1 deep research
- `19-COMPONENT_CUSTOMIZATION_PLAN.md` (30 KB) — Falcon library + existing scaffolding audit
- **`20-MAIN_BRANCH_FIDELITY_PLAN.md`** — this PDF's source notes

Falcon Specs v5.0 · Add Client Main-Branch Fidelity Plan
Generated 2026-05-16 by the Falcon Brain · origin/main read directly · zero backend changes
Code-ready. Say "go".